Introduction to Algorithm Design

An algorithm is a procedure to accomplish a specific task. An algorithm is the idea behind any reasonable computer program (dare I say even unreasonable ones)

To be interesting, an algorithm must solve a general, well-specified problem. It is important to note that an algorithmic problem is clearly distinct from an instance of that problem and understanding this distinction is both **Crucial And Fundamental**. Let's understand this distinction using C programming.

The Analogy: The Function Vs Function Call

1. The Problem (The Blueprint)

In C, think of the algorithmic "problem" as the **Function Signature And Its Documented Goal**.

Let's say the problem is writing a function to add two numbers!

```
// PROBLEM
// Input: Any two integers (a and b)
// Output: The sum of those two integers
int add(int a, int b)
{
    // algorithm goes here...
}
```

Notice that in the "Problem", **there are no actual numbers yet**. It is just a blueprint. It describes the universe of everything that is allowed to go in (any integer), and exactly what must come out (the sum).

2. The Instance (The Reality)

An "instance" is what happens when you actually run

the code and pass real, physical data into that function.

- Instance 1: add(5, 10);
- Instance 2: add(-2, 2);
- Instance 3: add(0, 0);

An instance is just one single scenario.

* **WHY THIS IS IMPORTANT**

Skiena is warning against biggest mistake junior developers make in coding interviews: **Solving the instance instead of solving the problem.**

Imagine you are asked to write the **add** function, and the interviewer says, "For example, if I give you 5 and 10, it should output 15". If you write code like this:

```
int add(int a, int b)
{
    return 15;
}
```

You successfully solved the instance and failed the problem entirely

THE PROBLEM: The abstract rules (Add any two integers)
THE INSTANCE: The specific test case (Add 5 and 10)

To be a correct algorithm your code cannot just work for one instance. It must perfectly solve **every possible instance that the problem allow.**

Another example is **Sorting**

PROBLEM: **Sorting**
INPUT: A sequence of n keys $a_1, \ldots, a_n$.

Output: The permutation (reordering) of the input such that $a'_1 \leq a'_2 \leq \ldots \leq a'_{n-1} \leq a'_n$

An instance of sorting might be an array of like {Mike, Bob, Sally, Jill} or a list of numbers. Determining that you're dealing with a general problem instead of an instance is your first step towards solving it.

An **algorithm** is a procedure that takes any of the possible input instances and transforms it to the desired output. There are many different algorithms that can solve the problem of sorting an example is **THE INSERTION SORT.**

***INSERTION SORT:** This method is an inplace, comparison based sorting algorithm that logically divides a continuous block of memory into a "sorted" left partition and an "unsorted" right partition, building the final sorted state one element at a time.

HOW IT WORKS: For every element in the unsorted partition, the algorithm performs three phases

1. **EXTRACTION:** It copies the target element into a temporary register to isolate it from memory mutations
2. **BACKWARD SCAN & SHIFT:** It scans the sorted left partition from right to left. Any element strictly greater than the payload is copied (shifted) one memory slot to the right, overwriting the adjacent space.
3. **THE DROP:** The shifting permanently halts the moment the scan encounters an element smaller than or equal to the payload (or hits the array boundary). The payload is then written into the correct opened memory slot.

CHECK C IMPLEMENTATION IN Insertion_sort.c

Note the generality of the algorithm. It works well on names as it does on numbers. Or anything else, given the appropriate comparison operation to test which of the two keys should appear first in sorted order. It can be readily verified that this algorithm correctly orders every possible input instance according to our definition of the sorting problem.